

Python para Solemne 3 - Cheat Sheet

Pandas: crear e inspeccionar

```
import numpy as np
import pandas as pd
```

```
df = pd.DataFrame(datos)
print(df.head())
print(df.shape)
print(df.dtypes)
print(df.isna().sum())
```

```
df.head()           primeras filas
df.shape            filas y columnas
df.dtypes           tipo de cada columna
df.isna().sum()     nulos por columna
```

Pandas: limpiar datos

```
cols = ["temperatura_K", "radio_Rsol",
        "distancia_pc", "magnitud_V"]
```

```
df_limpio = df.dropna(subset=cols).
copy()
```

```
dropna(subset=cols)  elimina filas con
                     nulos en
                     columnas clave
```

```
copy()              crea una copia
                     para modificar
                     sin ambigüedad
```

```
len(df)             cantidad de filas
```

Pandas: columnas derivadas

```
df_limpio["luminosidad_rel"] = (
    df_limpio["radio_Rsol"]**2 *
    (df_limpio["temperatura_K"] /
     5778)**4
)
```

Las operaciones con columnas se aplican fila a fila.

Pandas: filtros

```
mask = (
    (df["distancia_pc"] < 20) &
    (df["magnitud_V"] < 0.5) &
    (df["luminosidad_rel"] > 5)
)

sub = df.loc[mask, ["nombre", "tipo"]]

\&                combinar
                  condiciones
|                 una condicion u
                  otra
df.loc[mask, cols]  filas filtradas y
                  columnas elegidas
(...) \& (...)      use parentesis en
                  cada condicion
```

Pandas: agrupar

```
resumen = df.groupby("tipo").agg({
    "nombre": "count",
    "temperatura_K": "mean",
    "distancia_pc": "median",
    "luminosidad_rel": "mean"
})

resumen = resumen.rename(columns={"
    nombre": "cantidad"})
resumen = resumen.sort_values("
    luminosidad_rel",

    ascending=False)
```

Pandas: graficar por grupo

```
for tipo in df["tipo"].unique():
    sub = df[df["tipo"] == tipo]
    plt.scatter(sub["temperatura_K"],
                sub["luminosidad_rel"],
                label=tipo)
plt.legend()
```

Algoritmos: usar copia

```
periodos = [12.4, 3.1, 8.7]
ordenados = bubble_sort(periodos.copy()
    ())
```

```
print(periodos)      # original
print(ordenados)     # ordenada
```

```
lista.copy()         evita modificar la lista
                     original
```

```
ordenados[0]         minimo si esta ordenada
```

```
ordenados[-1]        maximo si esta ordenada
```

```
len(lista)           cantidad de elementos
```

Mediana con lista ordenada

```
n = len(ordenados)
```

```
if n % 2 == 1:
    mediana = ordenados[n//2]
else:
    mediana = (ordenados[n//2 - 1] +
                ordenados[n//2]) / 2
```

Buscar e interpretar

```
pos = binary_search(ordenados, 14.2)
```

```
if pos == -1:
    print("no encontrado")
else:
    print("posicion:", pos)
```

Python para Solemne 3 - Cheat Sheet

Bubble Sort

```
def bubble_sort(lista):  
    n = len(lista)  
    for i in range(n - 1):  
        for j in range(n - 1 - i):  
            if lista[j] > lista[j +  
1]:  
                lista[j], lista[j + 1]  
= (  
                    lista[j + 1],  
lista[j]  
                )  
    return lista
```

Ordena comparando vecinos. Use una copia si quiere conservar la lista original.

Binary Search

```
def binary_search(lista_ordenada,
objetivo):
    low, high = 0, len(lista_ordenada)
    - 1
    while low <= high:
        mid = (low + high) // 2
        if lista_ordenada[mid] ==
objetivo:
            return mid
        elif objetivo < lista_ordenada
[mid]:
            high = mid - 1
        else:
            low = mid + 1
    return -1
```

Solo es valida sobre una lista ordenada.

Ajuste con `np.polyfit`

```
import numpy as np

a, b, c = np.polyfit(d, R, deg=2)
R_fit = a*d**2 + b*d + c

deg=1      modelo lineal: m*x + b
deg=2      modelo cuadratico: a*x**2 +
           b*x + c
polyfit     retorna coeficientes desde mayor
           grado
```

Residuos y R^2

```
residuos = R - R_fit
SS_res = np.sum(residuos**2)
SS_tot = np.sum((R - np.mean(R))**2)
R2 = 1 - SS_res/SS_tot
```

residuos	dato medido menos modelo
R2 cercano a 1	ajuste explica bien los datos

Graficar ajuste

```
x_fino = np.linspace(0, 6, 200)
y_fino = a*x_fino**2 + b*x_fino + c

plt.scatter(d, R, label="datos")
plt.plot(x_fino, y_fino, label="ajuste")
plt.legend()
plt.grid(True)
```

Raíces con np.roots

```
# Resolver  $a*x**2 + b*x + c = 0$ 
raices = np.roots([a, b, c])

# Resolver  $a*d**2 + b*d + c = 12$ 
raices = np.roots([a, b, c - 12])
```

[a,b,c] coeficientes en orden descendente
c - 12 mover el umbral al lado izquierdo
np.roots puede entregar raices complejas

Filtrar raíces físicas

```
soluciones = []
for r in raices:
    if abs(r.imag) < 1e-10 and 0 <= r.
    real <= 6:
        soluciones.append(r.real)

r.imag           parte imaginaria
r.real           parte real
0 <= r.real <= 6 rango fisico o medido
```

Errores comunes

and	en pandas use \& con parentesis
binary_search(lista)	debe ser lista ordenada
np.roots([c,b,a])	orden incorrecto
tomar toda raiz	filtre complejas y no fisicas